

Abstracto

El objetivo del presente documento es el de brindar las bases importantes, bases que se deben de tomar en cuenta para el desarrollo de las aplicaciones, desde la realización de la documentación hasta la finalización del desarrollo. Se establecerán las reglas para los 3 esquemas que se manejan actualmente: Base de Datos, Back-end y Front-end.

Contenido

1	BackEnd	1
1.1	Etapas de desarrollo	1
2	Front End	3
3	Referencias a documentos y conexiones	4
3.1	Referencias	4
3.2	Conexiones	4

1

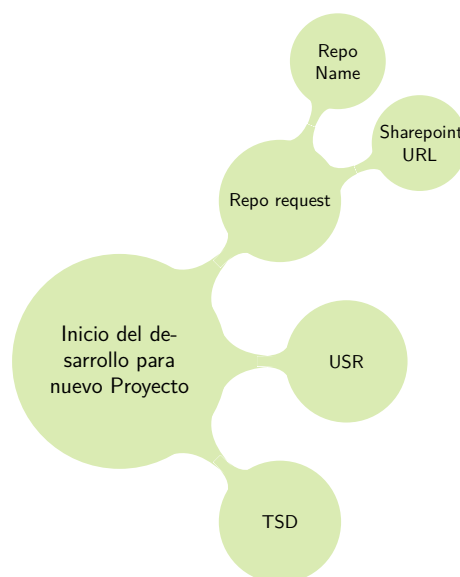
BackEnd

A continuación se establecen los puntos que el desarrollador debe tomar en cuenta para la implementación, ya sea de proyectos nuevos, refactorizaciones o agregar nuevas características a proyectos existentes.

1.1

Etapas de desarrollo

- Cuando se trate de un **proyecto nuevo**, el desarrollador deberá tener iniciado el TSD y tener iniciado el USR, al menos con los datos importantes del proyecto. El desarrollador deberá guardarlos en la carpeta de la documentación (sharepoint) junto con el BRD y el TSD y deberá mandar un correo a joel_castillejos3@jabil.com solicitando la creación del repositorio adjuntando la URL del repositorio de documentos y el nombre del repositorio a crear. El repositorio deberá tener una nomenclatura similar a este ejemplo: BreakoutPanelAndScrap.Americas.GTP



- Cuando se trate de agregar características a un proyecto se debe generar la parte inicial del TSD y agregarlo en el sharepoint con la nueva versión del proyecto a mejora (dentro del sharepoint el BSA a cargo debe de crear una carpeta con el nombre de la versión correspondiente).
- Para las remediaciones de código se deberá generar el IT Form Control Change y agregarlo a la carpeta correspondiente del proyecto.
- Para establecer un manejo adecuado de las ramas, siempre deberá crear la rama "development" a partir de "master" y todas las nuevas características o remediaciones deberán partir de la rama "development".
- Cuando se termine una remediación o una nueva característica se creará un "Pull Request" hacia development pidiendo por correo la revisión de código adjuntando la URL del Pull Request, el formato del Code Peer Review, la rama feature/hotfix deberá ser borrada una vez aprobado y completado el Pull Request.

- Toda cadena de conexión deberá ser encriptada con la ayuda de la herramienta JabilEncryptor y se establecerá el valor de `IsEncrypted` en `True`.
- Cuando se trate de un proyecto con arquitectura Rest Api (Api con JabilCore) se deberá configurar las cadenas de conexión en los 3 ambientes: Development, Staging y Prodduction. En cuanto a producción, el BSA nos deberá proporcionar en qué servidor se estará subiendo la aplicación.
- Cuando se trate de un proyecto con arquitectura Rest (Api con JabilCore) se deberá configurar la sección "JabilCore" en los 3 ambientes: Development (ya se encuentra configurado), Staging (el desarrollador establecerá el servidor y la estructura de carpetas) y Production (el BSA deberá proporcionar el nombre del servidor y el desarrollador establecerá la misma estructura de carpetas).
- Al finalizar un proyecto, el desarrollador tendrá que generar el manual de instalación en caso de que no se cuente con uno.
- Para los proyectos Rest (Api con JabilCore) los controladores deberán de estar lo más limpios posibles, es decir, tener el llamado al servicio y a lo mucho establecer el usuario que está haciendo la petición.
- Toda llamada del servicio en el controlador deberá estar dentro de una respuesta Ok como se muestra a continuación.

```
1 [HttpGet("{page}/{pageSize}/{predicate}/{sort}/{sortDirection}"), ActionName("GetSites"), Logging]
2 public IActionResult GetSites(int page, int pageSize, string predicate, string sort, string
   sortDirection)
3 {
4     var prueba = HttpContext.User.Identity.Name;
5     return Ok(this._SiteRetrieveService.RetrieveResultWhere(predicate, page, pageSize, sort,
   sortDirection, null));
6 }
```

- Todo servicio llamado desde un `ProcessService`, `RetrieveService` o `WriteService` debe ser verificado para determinar si se ejecutó de forma adecuada.

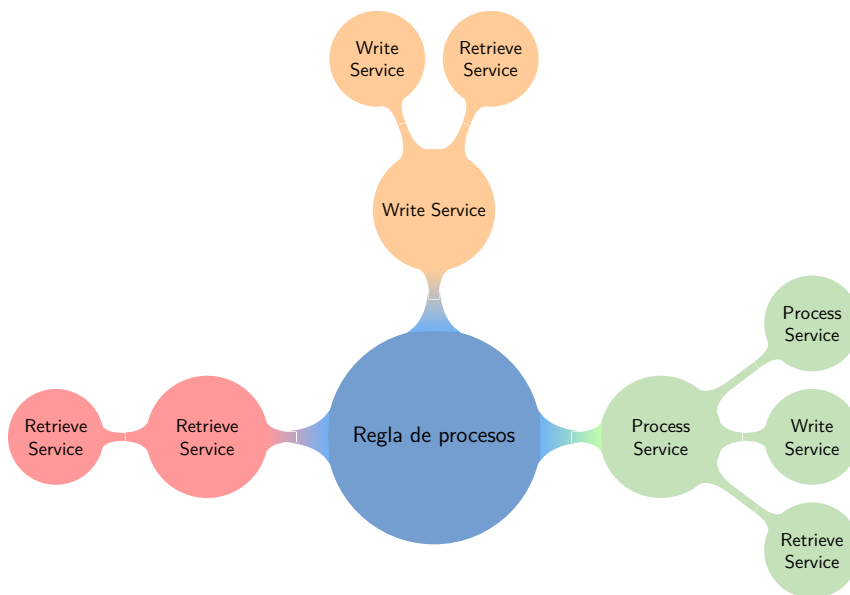
```
1 var nameResult = this._BuildingRetrieveService.RetrieveResultWhere(p => p.Name.ToLower().Equals(
   this._Entity.Name.ToLower())
2 p.SiteId == this._Entity.SiteId);
3 if (nameResult.Fail throw new OperationCanceledException(nameResult.GetMessage());
```

- Todos los nombres de los métodos deben tener la nomenclatura PascalCase y será nombrado en inglés.
- Todo comentario explicativo deberá ser en inglés
- Las entidades deberán cumplir con la nomenclatura PascalCase sin llevar algún caracter especial. en caso de que la tabla tenga una nomenclatura diferente en .NET deberá hacerse uso de las `DataNotation` para hacerlas coincidir.
- Para la validación de los registros en los procesos de escritura, deberán ser agregados en los métodos `ValidateCreate` y `ValidateUpdate` como se muestra a continuación.

```
1 public override bool ValidateCreate()
2 {
3     var nameResult = this._BuildingRetrieveService.RetrieveResultWhere(p => p.Name.ToLower().Equals(
   this._Entity.Name.ToLower() && p.SiteId == this._Entity.SiteId);
4     if (nameResult.Fail) throw new OperationCanceledException(nameResult.GetMessage());
5
6     if (nameResult.Data.ToList().Count > 0) throw new SystemValidationException("BUILDING.
   NAME_ALREADY_EXISTS");
7
8     return true;
9 }
10
11 public override bool ValidateUpdate()
```

```
12 {  
13     var nameResult = this._BuildingRetrieveService.RetrieveResultWhere(p => p.Id != this._Entity.Id  
14         && p.Name.ToLower().Equals)  
15     if (!nameResult.Success) throw new OperationCanceledException(nameResult.GetMessage());  
16     if (nameResult.Data.ToList.Count > 0) throw new SystemValidationException("BUILDING.  
17         NAME_ALREADY_EXISTS");  
18 }
```

- Respecto a la regla de los procesos, los RetrieveService sólo pueden llamar a otros RetrieveService, los WriteService sólo pueden llamar a otros RetrieveService o WriteService y los ProcessService pueden ejecutar cualquiera de los tres servicios.



- Cuando un proceso lleve demasiadas líneas de código o demasiada lógica o llamado a otros servicios entonces se deberá usar un ProcessService de forma obligatoria.
- Todo código comentado deberá ser borrado.
- Las enumeraciones deben estar en el proyecto de Models dentro de una carpeta llamada Enumerations, los DTO deberán estar en una carpeta llamada DTOs dentro de Models y las clases parciales que extienden a los modelos deberán estar en una carpeta llamada Partials .

2

Front End

A continuación se establecen los puntos que el desarrollador debe tomar en cuenta para su desarrollo con interfaces de usuario, ya sea con MVC Razon, Angular o ASP.NET .

- Cuando se requiera diseñar pantallas de un proyecto existente, el desarrollador deberá tomar en cuenta las pantallas existentes, para saber qué componentes y estructuras está usando.
- Cuando sea una aplicación nueva se tomará como ejemplo los mock ups presentados por el BSA, el desarrollador puede proponer ajustes al estilo.

- Al utilizar la plantilla Angular de Jabil todos los catálogos deberán llevar el mismo formato que el componente de ejemplo, llamado `console-component`.
- Si se utiliza la plantilla de Angular de Jabil deberá configurarse desde un principio los archivos index de cada ambiente.
- En caso de utilizar alguna autenticación como Okta/Cognito, el desarrollador deberá realizar las pruebas necesarias para el ambiente de producción.
- Los métodos en TypeScript debe utilizar la notación lower camel case .

- Es obligatorio utilizar servicios para comunicarse con las APIs al utilizar la plantilla de Angular.
- Todas las entidades deberán ser mapeadas como interfaces o clases en Angular para forzar el tipado.
- No repetir nombres claves para las llaves en los archivos de traducción.
- Sólo se permite un nivel de anidación:

```
1  "HOME": {  
2    "TITLE": "Hello Translate!",  
3    "SELECT_LANGUAGE": "Change Language",  
4    "ALERTS": "Alerts",  
5    "HELP": "Help"  
6  },  
7  "NAV": {  
8    "APPLICATIONS": "Application",  
9    "CATALOGS": "Catalogs",  
10   "CONSOLES": "Consoles"  
11 },  
12 "ERRORS": {  
13   "NOT_FOUND": "PAGE NOT FOUND",  
14   "DEFAULT": "There was an error in the application..."  
15 }
```

3

Referencias a documentos y conexiones

3.1

Referencias

Documento / App	Link
TSD	TSD Thecnical Specification Document - Template
Code Peer Review	Peer Review Template
IT Form Control Change	IT Form Control Change - Template
Jabil Encryptor	JabilEncryptor.zip - Repos (Azure)
USR	USR -Template

3.2

Conexiones

RDS Postgres

- Producción

```
1  Instancia de Lectura: "rds00-475137724643-ehsdg-001-instance-1-us-east-1a.c0lgirv4jfz4.us-east-1.  
   rds.amazonaws.com"  
2  Instancia de escritura: "rds00-475137724643-ehsdg-001-instance-1-us-east-1b.c0lgirv4jfz4.us-east  
   -1.rds.amazonaws.com"
```

- Desarrollo

```
1  "rds-50354-00029-emsmxgdl-001.cluster-c3djzrupnnjg.us-east-1.rds.amazonaws.com"  
2  "
```

Rabbit

```
1 "JabilLoggin": {  
2   "Host": "awuwe2web82",  
3   "UserName": "jlogging",  
4   "Password": "JabilLogging2348",  
5   "Port": 5672  
6 }
```

